

|                                           |           |
|-------------------------------------------|-----------|
| <b>Les structures « struct »</b>          | <b>2</b>  |
| Déclaration d'une structure               | 3         |
| Première méthode                          | 5         |
| Deuxième méthode                          | 6         |
| Troisième méthode                         | 7         |
| Première méthode                          | 8         |
| Imbrication des structures                | 9         |
| Initialisation d'une structure            | 10        |
| Opérateurs sur les structures             | 11        |
| Accès aux membres des structures          | 11        |
| Exemple                                   | 11        |
| Affectation de structures                 | 12        |
| Exemple                                   | 12        |
| Comparaison de structures                 | 12        |
| Tableaux de structures                    | 13        |
| Exemple                                   | 13        |
| Exemple                                   | 14        |
| <b>Renommer un type (Types synonymes)</b> | <b>15</b> |
| <b>Les unions</b>                         | <b>19</b> |
| <b>Les types énumérés</b>                 | <b>27</b> |

## LES STRUCTURES « STRUCT »

Une structure permet de regrouper plusieurs éléments éventuellement de types différents sous un seul identificateur.

C'est un moyen commode pour construire un type de données complexes.

Chaque élément de la structure définit un membre de la structure.

Un membre peut être de type simple ou structuré.

Une structure est un objet composé d'une séquence **fini** de membres, de types divers, portant des noms.

| <u>LANGAGE ALGO</u>              | <u>LANGAGE C</u>              |
|----------------------------------|-------------------------------|
| <b>Enregistrement, Article</b>   | <b>Structure</b>              |
| <b>Champ d'un enregistrement</b> | <b>Membre d'une structure</b> |

## Déclaration d'une structure

```
struct [nom_struct] {
```

```
    Déclarations ;
```

```
} [var1, ..., varm] ;
```

```
struct [nom_struct] {
```

```
    Type1 membre1 ;
```

```
    ...
```

```
    TypeN membreN ;
```

```
    } [var1, ..., varm] ;
```

Le nom de la structure est facultatif.

Chaque ligne de la partie déclarations définit un membre de la structure.

Remarque : tout comme les tableaux, le nombre de membres d'une structure est fixe.

## Exemple

```
struct date {
```

```
    short jour ;
```

```
    short mois ;
```

```
    int annee ;
```

```
};
```

Il y a plusieurs méthodes possibles pour déclarer des structures.

## Première méthode

```
struct vache{  
    char nom[DIM1]; //membre nommé nom tableau de 20 caractères  
    double poids ;    //membre nommé poids : un réel  
    int num ;          //membre nommé num un entier  
};
```

### Déclaration des variables

```
struct vache v1, v2 ;
```

Déclaration de deux variables de type struct vache de noms v1 et v2.

## Deuxième méthode

Déclaration des variables de type structure sans utiliser d'étiquette de structure

```
struct {  
    char nom[DIM1]; //membre nommé nom tableau de 20 caractères  
    double poids ;    //membre nommé poids : un réel  
    int num ;          //membre nommé num un entier  
} v1, v2;
```

L'inconvénient de cette méthode est qu'il sera par la suite impossible de déclarer une autre variable du même type.

En effet,  
**struct {**

```
char nom[DIM1]; //membre nommé nom tableau de 20 caractères  
double poids ;    //membre nommé poids : un reel  
int num ;          //membre nommé num un entier  
} v3 ;
```

Les deux structures ont beau avoir le même nombre de membres, avec les mêmes noms et les mêmes types, elles seront considérées de types différents.

Il sera impossible d'écrire  $v3 = v1$  ;

### Troisième méthode

On peut combiner déclaration d'étiquettes de structure et déclaration de variables, comme ceci :

```
struct vache{
```

```
    char nom[DIM1]; //membre nommé nom tableau de 20 caractères  
    double poids ;   //membre nommé poids : un reel  
    int num ;        //membre nommé num un entier  
} v1, v2;
```

Déclare les deux variables v1 et v2 et donne le nom vache à la structure.

On pourra utiliser ultérieurement le nom struct pour déclarer d'autres variables :

```
struct vache v3, v4 ;
```

// v3 et v4 sont du même type que v1 et v2.

### **Recommandation**

De ces trois méthodes c'est la première qui est recommandée, car elle permet de bien séparer la déclaration du type structuré de ses utilisations.

### Première méthode

```
struct vache{
```

```
    char nom[DIM1]; //membre nommé nom tableau de 20 caractères  
    double poids ;   //membre nommé poids : un reel  
    int num ;        //membre nommé num un entier  
} ;
```

### **Déclaration des variables**

```
struct vache v1, v2 ;
```

Déclaration de deux variables de type struct vache de noms v1 et v2.

## Imbrication des structures

|                                                                    |                                                                                      |
|--------------------------------------------------------------------|--------------------------------------------------------------------------------------|
| <pre>struct point{<br/>    double x;<br/>    double y;<br/>}</pre> | <pre>struct rectangle{<br/>    struct point p1, p2;<br/>    int couleur;<br/>}</pre> |
|--------------------------------------------------------------------|--------------------------------------------------------------------------------------|

```
#define NB_SOM_MAX 100
```

```
struct ligne{  
    int nbsommets ;  
    struct point sommets[NB_SOM_MAX] ;  
}
```

## Initialisation d'une structure

Une structure peut être initialisée par une liste d'expressions constantes à la manière des initialisations de tableau.

## Exemple

```
struct date d={6, 9,2022} ;
```

```
struct point m1 = {3.0, 3.0}, m2={10.5, 10.5} ;
```

```
struct rectangle r={m1, m2, 2} ;
```

```
struct ligne l={2, {m1, m2}} ;
```

## Opérateurs sur les structures

### Accès aux membres des structures

#### Syntaxe

IndentificateurVar.MembreStructure

Pour désigner un membre d'une structure, il faut utiliser l'opérateur de sélection de membre qui se note . (point).

#### Exemple

```
struct vache v1, v2 ;
```

```
v1.nom      /* désignera le membre nom de v1 */
```

```
v1.num      /* désignera le membre num de v1 */
```

```
v1.nom[0]   /* désignera l'accès au premier caractère  
             du nom de v1*/
```

## Affectation de structures

On peut affecter une structure à une variable structure de même type, grâce à l'opérateur d'affectation (=)

#### Exemple

```
struct vache v1,v2 ;
```

```
...
```

```
v1 = v2;
```

## Comparaison de structures

Aucune comparaison n'est possible sur les structures, même pas les opérateurs == et !=.

## Tableaux de structures

Une déclaration de tableau de structures se fait selon le même modèle que la déclaration d'un tableau dont les éléments sont de type simple.

### Syntaxe

**TypeStructure identificateurTab[DIM] ;**

### Exemple

Supposons que l'on ait déjà déclaré la struct personne, si on veut déclarer un tableau de 100 structures de ce type, on écrira :

```
#define DIM 100
struct vache tabVache[DIM];
```

Pour référencer le nom de la vache qui occupe la  $(i+1)^{\text{ème}}$  position dans tab on écrira :

**tabVache[i].nom.**

## Exemple

```
#define NB_SOM_MAX 100
```

```
struct ligne{
    int nbsommets ;
    struct point sommets[NB_SOM_MAX] ;
} ;
```

```
struct ligne l1 ;
```

```
...
```

```
l1.sommets[i] //accès au  $(i+1)^{\text{ème}}$  sommet de la ligne l1.
```

## RENOMMER UN TYPE (TYPES SYNONYMES)

**Syntaxe :**

```
typedef    ancienNom    nouveauNom ;
```

L'instruction **typedef** permet de renommer un type.

Elle est utilisée généralement pour donner un nom relativement court et surtout plus explicite à un type construit ou à un type de base existant.

Le nouveau type ainsi nommé sera utilisé par la suite de la même manière qu'un type de base du Langage C.

## Exemple

```
typedef int Entier ;
```

Définit un type synonyme appelé entier ayant les mêmes caractéristiques que le type prédéfini **int**.

Une fois cette définition réalisée, nous pouvons utiliser ce nouveau type pour définir des variables et nous pouvons mélanger les variables de ce type avec des variables entières pour réaliser des expressions.

```
Entier e1=23, e2=5, tab[DIM]={1,2,3,4,5,6,7};  
int i;  
i = e1 + e2;  
tab[20] = i - 60;
```



Dans le cas de la déclaration de tableaux, la taille du nouveau type se trouve après le nouveau nom de type.

```
typedef char Chaine[DIM] ;
typedef double Vecteur[DIM];
typedef int Tableau[DIM];
struct vache TabVache[DIM];
```

**Tableau tab ;**

**Vecteur v1, v2 ;**

Dans cet exemple, Tableau devient un type synonyme correspondant au type tableau de 10 int.

La variable tab est un tableau de 10 entiers.

**Attention :**

typedef ne réserve pas d'espace mémoire.

Les noms sont des types ; ils sont donc inaccessibles comme variables.

L'utilisation de typedef permet de faciliter les définitions de prototypes pour permettre aux programmeurs d'écrire du code plus lisible.

**typedef struct point Point ;**

Il est aussi possible de définir un type équivalent à une structure :

```
typedef struct{
    short jour ;
    short mois ;
    int annee ;
} Date ;
```

```
typedef struct{
    char nom[20] ;
    double poids ;
    int num ;
} Vache ;
```

Et d'utiliser directement ce type.

Ceci évite les répétitions fastidieuses du mot réservé struct.

Point pt ;

Date d1, d2 ;

Personne p1, p2 ;

# LES UNIONS

## Définition

L'union permet de définir une variable qui pourra contenir des éléments de type différent au cours du temps.

La liste de déclarations ne définit pas l'ensemble des membres de l'union mais une liste de choix possible.

## Syntaxe

```
union [nomUnion] {  
    déclarations ;  
} [var1, ...varn];
```

La syntaxe est  
similaire à la syntaxe  
définissant une  
structure.

```
union [nomUnion] {
```

```
    Type1 membre1 ;
```

```
    ...
```

```
    TypeN membreN ;
```

```
} [var1, ...varn];
```

## Exemples

|                                                                      |                                                                                                                                                                      |
|----------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| a) typedef union {<br><br>int i ;<br><br>float f ;<br><br>} Nombre ; | b) typedef union {<br>struct {<br>int abscisse ;<br>int ordonnee ;<br>} Cartesienne ;<br>struct {<br>double rho ;<br>double theta ;<br>} Polaire ;<br>} Coordonnee ; |
|----------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|

## Déclaration

Coodonnee p1, p2 ;

Nombre n ;

n est une variable qui possède soit une valeur entière,  
soit une valeur flottante, mais pas les deux à la fois.

## Accès aux membres de l'union

L'accès se fait avec le même opérateur de sélection (noté .) que celui qui sert à accéder aux membres des structures.

**n.i = 10 ;        //n possède une valeur entière**

**n.f = 10.559 ;    //n possède une valeur flottante**

### Remarques

Les deux variables n.i et n.f occupent la même place en mémoire donc une seule donnée.

Avec la structure struct au lieu de la structure union, il y aurait eu deux emplacements différents.

**La taille de n est celle du plus grand membre de l'union.**

## Problème

\* Comment savoir à tout moment si on doit utiliser un entier ou un flottant ?

\* Comment savoir à tout moment si on doit utiliser les coordonnées cartésiennes ou polaires.

## Solution

**Association de l'union à un indicateur (variable) pour indiquer le membre de l'union qui est valide.**

**En pratique, une union et son indicateur sont généralement englobés à l'intérieur d'une structure.**

```
typedef struct{
```

```
    TypeNombre indNB ; //indicateur de la nature du nbre
```

```
    union {
```

```
        int i ;
```

```
        float f ;
```

```
    }nb ;
```

```
} Nombre ;
```

Cette structure a deux membres :

- indNB de type int et
- nb de type union (d'int et de float).

**Déclaration des variables par :**

**Nombre n1, n2 ;**

**Puis on pourra utiliser de la manière suivante :**

**n1.indNB = ENTIER ;**

**n1.nb.i = 3 ;**

**ou**

**n2.indNB = FLOTTANT ;**

**n2.nb.f = 3.14159 ;**

```
typedef struct{
```

```
    TypeCoord icoord ; //indicateur de la nature des coords
```

```
    union{
```

```
        struct{
```

```
            int abscisse ;
```

```
            int ordonnee ;
```

```
        } Cart ;
```

```
        struct{
```

```
            double rho ;
```

```
            double theta ;
```

```
        } Pol ;
```

```
    }val ;
```

```
} Coordonnee;
```

## Déclarations

**Coordonnee p1, p2 ;**

p1. typ\_coord = CARTESIENNE ;

p1.val.cart.abscisse

p1.val.cart.ordonnee

p2. typ\_coord =POLAIRE;

p2.val.pol.rho

p2.val.pol.theta

# LES TYPES ÉNUMÉRÉS

## Définition

C'est un type qui ne prend des valeurs que dans un ensemble précis de valeurs constantes.

C'est un moyen pratique de définir un ensemble fini de valeurs constantes pour une variable, par exemple un ensemble d'état.

## Déclaration

```
enum [Identificateur_type] {liste d'indentificateurs} [liste variables];
```

Le rôle de `Identificateur_type` est le même que pour `struct`.

**IL sert à déclarer des variables enum.**

## Exemple

```
enum Typ_Entier {ENTIER, FLOTANT} ;
```

```
enum Typ_Coord {CARTESIENNE, POLAIRE} ;
```

```
enum Jour {LUNDI,..., DIMANCHE} ;
```

```
enum Bool {FALSE, TRUE} ;
```

Déclaration des variables

```
enum Jour j ;
```

```
enum Typ_coord nat_coord=POLAIRE ;
```

Ils servent à rajouter du sens à de simples numéros, à définir des variables qui ne peuvent prendre leur valeur que dans un ensemble fini de valeurs possibles identifiées par un nom symbolique.

## RENOMMAGE

```
typedef enum {FALSE, TRUE} Bool ;
```

```
Bool trouve=FALSE ; /*Déclaration d'une variable
```

```
Booléenne*/
```

### Remarque

**Le premier identificateur constant reçoit la valeur 0, le suivant la valeur 1 et ainsi de suite sauf si le programmeur spécifie des valeurs précises.**

**FALSE a la valeur 0, TRUE la valeur 1.**

## Exemple

```
typedef enum {JANVIER=1, ..., DECEMBRE} Mois;
```

FEVRIER a la valeur 2, Mars la valeur 3,...,

DECEMBRE la valeur 12.

On peut aussi avoir des valeurs non séquentielles

```
typedef enum {LUNDI='L', MARDI='m',  
ERCREDI='M', JEUDI='J', VENDREDI='V',  
SAMEDI='S', DIMANCHE='D'} Jour ;
```

## Exemples

```
typedef enum {noir , bleu , vert ,  
rouge , blanc , jaune } Couleurs;
```

Les valeurs générées par le compilateur seront :

```
noir = 0    bleu =1    vert =2  rouge=3    blanc =4  
jaune=5
```

```
typedef enum {noir = -1, bleu ,  
vert , rouge = 5, blanc , jaune }  
Couleurs;
```

Les valeurs générées par le compilateur seront :

```
noir = -1    bleu =0    vert =1  rouge=5    blanc =6  
jaune=7
```